dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Computers

Sanjiang Li *

***Abstract*—Distributed quantum computers execute a circuit across small quantum processing units linked by entanglement. Their compiler must minimise consumption of Einstein–Podolsky–Rosen (EPR) pairs while still routing large circuits quickly and reliably on hardware with finite communication ports and finite core capacity. We present dSABRE, a distributed extension of the widely used SABRE routing principle. Rather than treating a teleport as a high-cost SWAP, dSABRE scores each inter-core teledata move as one macro-action: the EPR pair it spends, the port staging it requires, the destination capacity it consumes, and the front-layer progress it provides with lookahead discounted by depth in the circuit's directed acyclic graph (DAG). A hard capacity floor on every teleport, together with a bounded recovery transaction, makes dSABRE provably complete: it routes every gate and never aborts at saturation, on any instance that passes a pre-routing check. Across 21 Munich Quantum Toolkit (MQT) Bench circuits with 25–64 logical qubits, dSABRE reduces geometric-mean EPR consumption by 49–62% relative to TeleSABRE. It completes all 45 evaluated circuit–architecture instances against TeleSABRE's 40, and compiles a 360-qubit QFT on twenty cores in 48 s. These results show that capacity-aware, feasibility-preserving extensions of a simple SABRE-style objective can provide effective, scalable, and robust distributed routing. Code is available at https://github.com/ebony72/dsabre.**

***Index Terms*—distributed quantum computing, circuit routing, qubit teleportation, SABRE, EPR minimisation, qubit mapping, lookahead heuristic**

I. Introduction

Distributed quantum computers (DQCs) combine several small quantum processing units (QPUs) through entanglement links [1]. They offer a practical route to scaling beyond a single processor, but introduce a compilation problem absent from monolithic devices: a two-qubit gate whose operands occupy different cores requires inter-core communication. In this work that communication is supplied by an Einstein–Podolsky–Rosen (EPR) pair. EPR generation is slow, noisy and rate-limited relative to local operations in current hardware [2], [3], so a router should avoid spending an EPR pair unless the move it enables is worthwhile.

Distributed compilation has three connected tasks: (i) allocate logical qubits to cores, (ii) choose how to realise a non-local interaction, and (iii) route and schedule the resulting local and inter-core operations [4]. Hypergraph partitioning addresses the first task by exposing global communication structure [5], [6]. An allocation alone, however, does not ensure that the computation can execute on a device with finite ports, finite core capacity, and non-trivial intra-core connectivity. During routing, a state may have to be moved to a communication port, that port may be occupied, and the target core may lack a spare physical slot. These are dynamic constraints of the evolving layout, not properties of the initial partition alone.

A practical routing objective. A useful compiler must be effective, efficient, and robust: it should reduce the expensive resource, compile instances at useful scale, and return a valid routed circuit. These requirements motivate a SABRE-style design. SABRE [7] is the layout and routing pass Qiskit [8] selects by default: it chooses a SWAP by the progress it makes on the *front-layer* two-qubit gates, with bounded lookahead. Its appeal is not global optimality but a simple, scalable decision rule that performs well in production compilation flows. The question addressed here is what that rule must price when the machine is several QPUs rather than one.

The answer is not to replace a SWAP by a teleport on a higher-weight edge. A SWAP is symmetric and preserves the occupancy of the regions it joins. A teleport is directional: it is feasible only after its source state reaches a communication port, and it consumes a free slot in the destination core. Thus, an inter-core move has three inseparable consequences: it spends an EPR pair, may require port staging before it is possible, and reduces the capacity left for future moves. Treating these effects as one macro-action is the unifying design principle of dSABRE.

dSABRE is a SABRE-style teledata router that scores a candidate macro-action—source staging, port eviction if needed, and one teleport—by the routing progress it buys and the resources it consumes. The progress term is SABRE's front-layer distance reduction plus a lookahead discounted by depth in the circuit's directed acyclic graph (DAG), the dependency graph defined in Section II-A. The two added costs are an availability term for staging at a port and a graded capacity term for the destination core: the two routing-state constraints introduced when a one-way teleport replaces a local SWAP. Latency, coherence, link contention, and fidelity are teleport-specific too, but lie outside the model optimised here (Section VI). The capacity floor is deliberately kept out of the score, as a hard admission test applied when candidates are generated (Section III-A): a weight can be outvoted by a large progress term, an admission test cannot. The score therefore ranks feasible progress-making moves, while the floor and the bounded recovery transaction keep finite capacity from becoming an unrecoverable mid-route dead end (Sections III-A–III-F).

Scope of communication. This paper deliberately optimises teledata, which moves one logical state to a new core and consumes its EPR pair immediately. Because no entanglement

S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).
*ORCID: 0000-0002-3332-2546.

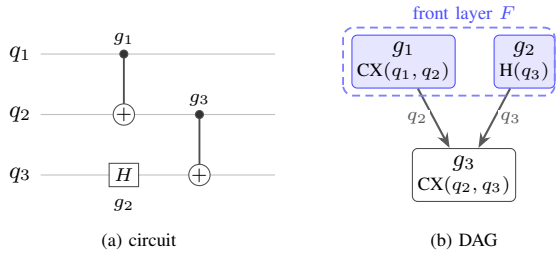
is retained, an EPR count does not depend on how long a link qubit can hold its state, and the occupancy of the cores is the only resource a route must manage over time—the capacity problem studied here. Telegate can be preferable when a group of compatible remote gates shares an entangled resource, but its benefit depends on gate grouping, link-buffer occupancy, and the lifetime of retained entanglement [9]: it is a distinct joint primitive-selection problem, not an omitted zero-cost candidate. dSABRE does not claim teledata dominates telegate, and identifies burst extraction and telegate selection as extensions (Section VI).

Scope of architecture and comparison. The algorithm is defined on an arbitrary *connected* coupling graph with designated ports and queries that graph through distance tables: each core's intra-core graph is connected and so is the core graph, so every distance the router queries is finite. Nothing further is assumed—not degree, regularity, diameter, or the number and placement of ports.

The evaluation exercises that generality over four architectures: the grid-of-grids devices of TeleSABRE and two networks of IBM heavy-hex cores, a ring and a star. The reported EPR reductions are established across all four, which differ in core degree, diameter, and port placement. Likewise, EPR counts are directly comparable only when methods use the same communication primitive, architecture, and cost accounting. TeleSABRE [10] is the primary head-to-head baseline because it shares the teledata setting and local-SWAP accounting. DMapS [11] and `pytket-dqc` [6], [12] count EPR pairs under a different primitive—a remote SWAP and a telegate, respectively—and under different assumptions about communication capacity and entanglement lifetime. Those two comparisons are therefore reported separately, each stating the assumptions its EPR count has to be read under.

Contributions.

- **A unified capacity-aware teledata score.** We formulate every inter-core move as a macro-action and extend SABRE's distance-progress objective with the two routing-state costs that distinguish teleportation from a SWAP: port availability and the destination capacity it consumes. The score uses front-layer distance reduction plus DAG-depth-discounted lookahead, and exhausts executable local front-layer work before considering an EPR-consuming action.
- **Feasibility-aware routing with completion conditions.** We give pre-routing capacity conditions, a hard legality floor, and a recovery transaction (Algorithm 2) for the saturated states in which the ordinary loop has no legal move that makes progress. Under these checkable conditions dSABRE routes every gate and does not abort; this is a completion guarantee, not a claim of EPR optimality. The measurements locate where it earns its place: no reported route among the 21 circuits at 25–64 qubits invokes recovery at all, whereas at scale it retires 6, 10, and 66 gates over the reported 100-, 200-, and 360-qubit routes.
- **Evidence of practical effectiveness and efficiency.** Across 21 Munich Quantum Toolkit (MQT) Bench circuits with 25–64 logical qubits, dSABRE reduces geometric-mean EPR consumption by 49–62% relative to TeleSABRE. It completes all 45 evaluated circuit–architecture instances, against TeleSABRE's 40. The implementation also compiles a 360-qubit QFT on twenty cores in 48 s, with compile time growing near-linearly in gate count at fixed architecture (Section IV-C). Ablations separate the effects of capacity handling, lookahead construction, and recovery, while the cross-model experiments distinguish like-for-like comparisons from results contingent on different EPR use assumptions.



(a) circuit (b) DAG

Fig. 1. DAG for a circuit with $\mathrm{CX}(q_1, q_2)$, $\mathrm{H}(q_3)$, and $\mathrm{CX}(q_2, q_3)$. The first two gates have no predecessors and form the initial front layer. The final CX becomes ready only after both have executed.

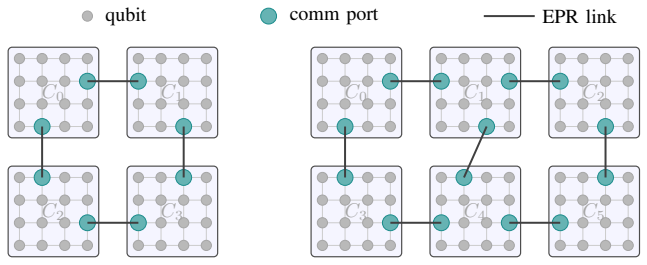
Section II gives the circuit, teleportation, architecture and SABRE background; Section III presents dSABRE and its completion argument; Section IV evaluates it; and Sections V–VI discuss related work and extensions. Supplementary material at the URL above carries the full completion proof, the complexity validation, per-circuit and per-seed counts, benchmark provenance, the parameter and cost-model sweeps, and the capacity, lifetime and action-level audits behind the cross-model comparisons.

II. Background

A. Circuits and routing

A quantum circuit is represented as a DAG: each node is a gate and each edge records a dependency induced by a shared logical qubit. The *front layer* is the set of dependency-free gates and can be executed in parallel when the hardware permits. This paper considers single-qubit (1Q) gates and two-qubit (2Q) gates, in particular CX gates. Together, CX and arbitrary 1Q unitaries are *exactly* universal: they decompose any n-qubit unitary [13], whereas a finite set such as $\{\mathrm{CX}, H, T\}$ is only *approximately* so. Hardware provides a finite set: the benchmark circuits of Section IV-A are transpiled into IBM's $\{\mathrm{CX}, R_z, \sqrt{X}, X\}$.

Let a circuit contain n logical qubits and let the target architecture have $P \geq n$ physical qubits. An initial layout is an injective mapping from logical to physical qubits. A 2Q gate is executable only when its operands are adjacent in the coupling graph. Routing therefore inserts local SWAPs and, in a distributed machine, inter-core communication operations; each local SWAP costs c_{swap} and each inter-core teleport c_{tele}. A SWAP exchanges the states of adjacent qubits and decomposes into three CX gates. Figure 1 illustrates the DAG and front-layer notion used by the router; *circuit depth* is the length of its longest path.



(a) 2×2 B-grid
4 cores, 64 qubits, 4 EPR links

(b) 2×3 H-grid
6 cores, 96 qubits, 7 EPR links

Fig. 2. Distributed architectures used in the experiments. Grey vertices are physical qubits, teal vertices are communication ports, and bold edges are inter-core EPR links. Each core is a 4×4 grid: (a) a 2×2 B-grid and (b) a 2×3 H-grid.

B. Teleportation

Quantum teleportation transfers a qubit state between two locations using a pre-shared EPR pair and classical communication; the state is removed from the sender and reconstructed at the receiver [14]. In this paper, one consumed EPR pair is one unit of inter-core communication cost, and "EPR" as a unit (e.g. "10 EPRs") denotes EPR-pair count.

Two communication primitives are relevant. *Teledata* moves a logical qubit to another core, after which subsequent gates may execute locally. *Telegate* implements a non-local 2Q gate without relocating either logical qubit [9]. Teledata can amortise its cost over a qubit's subsequent residence in the destination core. Telegate can amortise one entangled state over a compatible group of remote gates, but this requires an available link qubit and entanglement that survives for the group duration; Section IV-E measures what those two assumptions are worth.

C. Distributed architecture

We model the distributed quantum computer as a weighted coupling graph $G_{\mathcal{A}} = (V_{\mathcal{A}}, E_{\mathcal{A}}, w)$. Its physical-qubit vertices are partitioned into K cores, $V_{\mathcal{A}} = V_1 \sqcup \cdots \sqcup V_K$. Intra-core edges E_{intra} model local connectivity and have unit weight. Inter-core edges E_{inter} join designated communication ports and have weight $w_{\text{link}} = 10$, the cost of a teleport relative to a unit intra-core hop. A communication port is an ordinary physical slot that also terminates an inter-core link; it may hold a data qubit when it is not used for communication. We assume throughout that $G_{\mathcal{A}}$ is *connected*, and in the stronger form the router actually requires: the subgraph induced by each V_i is connected, and so is the core graph on $\{1, \ldots, K\}$ induced by E_{inter}. Every distance below is then finite.

dSABRE uses three distance tables: physical distance d_{phys}, intra-core distance d_{intra}, and core-graph distance d_{core}. It therefore applies to arbitrary connected intra-core graphs and inter-core topologies. Our experiments instantiate the grid-of-grids architectures of TeleSABRE [10], shown in Figure 2, and the networks of IBM heavy-hex cores of Section IV-C.

Physical distances need not be stored in a dense all-pairs table: every inter-core path exits and enters through ports, so with Π_i the ports of core i, for $p \in V_i$ and $q \in V_j$,

$$d_{\text{phys}}(p,q) = \min_{\pi \in \Pi_i,\, \pi' \in \Pi_j} \big[d_{\text{intra}}(p,\pi) + d_{\text{port}}(\pi,\pi') + d_{\text{intra}}(\pi',q) \big]. \tag{1}$$

The implementation memoises these queries.

D. SABRE-style routing

SABRE [7] is a heuristic router that repeatedly executes ready gates and inserts a candidate SWAP when a front-layer 2Q gate is not adjacent. Its score favours a SWAP that reduces the distance of front-layer gates while also considering a bounded lookahead set. We use the LightSABRE form [15]:

$$H = \frac{1}{|F|} \sum_{g \in F} \Delta(g) + w_e \frac{1}{|E|} \sum_{g \in E} \Delta(g), \tag{2}$$

where F is the front layer, E is the extended set, w_e weights the lookahead term against the front-layer term, and $\Delta(g) = d^g_{\text{new}} - d^g_{\text{old}}$ is the distance change for gate g. A negative value denotes progress. dSABRE retains this front-layer-plus-lookahead principle, but adapts the candidate action and score to account for port staging and finite capacity in the destination core. Building on a heuristic that production toolchains already run keeps compile time practical and the output executable: the router carries the layout with it and admits only moves that layout allows. What remains open is what the SABRE objective must become when one chip becomes several.

III. dSABRE

dSABRE is a SABRE-style router for multi-core processors. It uses teledata alone: one teleport consumes one EPR pair and moves one logical qubit across one adjacent-core link. Thus, unlike an amortised telegate, no entanglement lifetime or live EPR buffer is modelled.

The routing state is a layout ℓ and inverse ℓ^{-1}, a DAG W of unexecuted gates, its front layer F of gates with no unexecuted predecessors, and a bounded extended set E beyond F. A front-layer 2Q gate is *intra-core* when both operands are in one core and *inter-core* otherwise. Table I lists the notation.

A. Routing Policy, Score, and Legality

dSABRE has two actions: an intra-core SWAP and an inter-core teleport. It first resolves non-adjacent intra-core front gates by local SWAPs. Only when none remain does it score teleports for inter-core front gates. This priority rule avoids imposing an artificial exchange rate between the local and global scorers.

A teleport may require port eviction and staging SWAPs, and consumes one free slot in its landing core. We treat these operations and the teleport as one *macro-action*: a SWAP is a single primitive, while a teleport is a short sequence scored and applied as one unit. *Atomic* below means that such a sequence either completes or leaves the state untouched. Recovery (Section III-F) has the same property, but is the fallback for when neither ordinary action makes progress rather than a third ordinary action. If $\Delta\Phi(a)$ is the change action a induces in a front-layer-plus-lookahead routing potential—negative when a

TABLE I
NOTATION FOR SECTION III. DEFAULTS ARE FIXED ACROSS ALL EXPERIMENTS AND ARCHITECTURES.

Symbol	Meaning	Default
ℓ, ℓ^{-1}	layout and inverse layout	—
W, F, E	remaining DAG, front layer, extended set	—
F_c, E_c	restrictions of F, E to core c	—
K, M, P, n	cores, qubits/core, physical, logical qubits	—
$d_{\text{intra}}, d_{\text{core}}$	intra-core and core-graph distance	—
D_M, D_K	intra-core and core-graph diameter	—
$\text{dep}(g)$	BFS depth of g in E	—
π_s, π_d, n_s	source and destination ports; staging slot	—
$d_{\text{prep}}, c_{\text{cap}}$	preparation cost, capacity penalty	—
$\Delta_F, \widetilde{\Delta}_E$	front-layer and lookahead score change	—
f_c, F_{free}	free slots in core c; total free slots	—
$c_{\text{swap}}, c_{\text{tele}}$	local-SWAP and teleport cost	3, 10
w_e	lookahead weight	0.25
L, γ	extended-set capacity and depth decay	20, 0.9
τ, c_{pen}	capacity threshold and penalty	3, 15
ϕ_1	legality floor (Eq. (4))	2
L_{deadlock}	stalled-iteration limit	$4D_K(D_M+1)$

leaves the waiting gates closer to executable—the conceptual objective is

$$s(a) = \underbrace{\text{cost}(a)}_{\text{resource spent}} + \underbrace{\text{pen}(a)}_{\text{capacity consumed}} + \underbrace{\Delta\Phi(a)}_{\text{progress bought}} . \quad (3)$$

All compared teleport candidates consume one EPR pair, so this constant is omitted from their ranking.

Legality rule: The capacity penalty ranks feasible moves; it does not determine feasibility. Feasibility is enforced by a separate *legality floor*: an ordinary teleport is generated only if its landing core c_{next} has

$$f_{c_{\text{next}}} \geq \phi_1 = 2, \quad (4)$$

the free slots being counted before the move. A teleport consumes one slot of its landing core, so after the arrival that core still retains at least $\phi_1 - 1 = 1$ free slot, and local SWAPs preserve every core's occupancy. Hence ordinary routing never creates a full core, whose occupied communication port could otherwise prevent a later departure. Equation (4) is used in candidate generation and in the recovery proof.

B. Workflow

Figure 3 summarises the routing loop and Algorithm 1 states it in full. The router retains a *progress checkpoint*—the layout and its inverse, the remaining DAG W, the *trace* of SWAPs, teleports and executed gates emitted so far, and the counters that summarise them—as they stood immediately after the last decrease in $|W|$, and initially the input layout, the full DAG and an empty trace. Restoring it rewinds the trace as well as the layout, so the two never disagree: the operations left in the trace are exactly those that carry the input layout to the current one. Each iteration is:

P1: Execute. Repeatedly execute front-layer 1Q gates and adjacent intra-core 2Q gates. Stop if W is empty.

P2: Classify. Partition the remaining front layer into non-adjacent intra-core gates F_{intra} and inter-core gates F_{inter}.

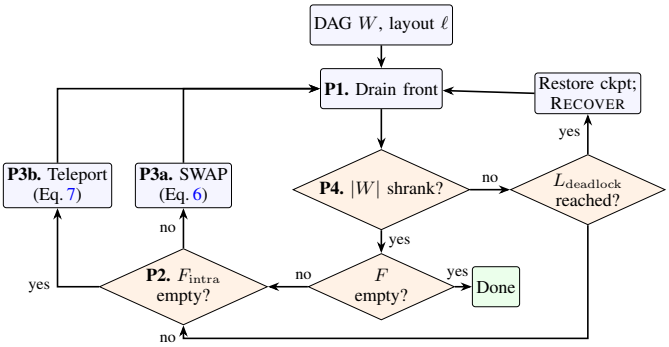


Fig. 3. dSABRE routing workflow. P1 drains executable gates; P2 classifies the remaining front layer; P3 applies either the best local SWAP or the best legal teleport macro-action; and P4 checkpoints every decrease in the remaining DAG. Three conditions invoke the guaranteed transaction of Section III-F: an empty legal candidate set, exhausted candidate attempts, and L_{deadlock} iterations without progress. Only the third is drawn; the two edges from P3b to the recovery node are omitted for clarity, and Lemma 2 in Section III-F shows the second never fires.

P3: Route. If $F_{\text{intra}} \neq \emptyset$, apply the lowest-score local SWAP of each active core. Otherwise enumerate legal teleport macro-actions for F_{inter} and try them in score order. Each attempt is atomic: a failed eviction, staging sequence, or teleport restores the pre-attempt state. If no candidate is legal or none commits, restore the progress checkpoint and invoke recovery; by Lemma 2 in Section III-F the second case cannot arise.

P4: Checkpoint or recover. A decrease in $|W|$ replaces the checkpoint and resets the stalled counter. Otherwise increment it; after L_{deadlock} stalled iterations, restore the checkpoint and invoke recovery. Recovery is atomic: it either retires its target gate and restores the reserve, or restores its entry state—a branch Theorem 1 in Section III-F rules out, leaving it an assertion on the hypotheses rather than an expected outcome.

C. Intra-Core SWAP Heuristic

For active core c, let F_c be its non-adjacent intra-core front gates and let $E_c \subseteq E$ contain the extended-set gates whose operands currently both reside in c. The global construction of E supplies one consistent depth $\text{dep}(g)$ for both scorers: the layer of g in a *breadth-first* (BFS) expansion of the remaining DAG, so a gate one dependency beyond the front layer has $\text{dep} = 1$, one beyond that $\text{dep} = 2$. For a candidate SWAP, define

$$\widetilde{\Delta}_E^c = \sum_{g \in E_c} \gamma^{\text{dep}(g)} \left(d_{\text{new}}^g - d_{\text{old}}^g\right), \quad (5)$$

where $0 < \gamma \leq 1$ and both distances are d_{intra}. The candidates are the physical edges (u, v) within c incident to an operand of a gate in F_c; the selected SWAP minimises

$$H_{\text{intra}} = \frac{\Delta_F^c}{|F_c|} + w_e \frac{\widetilde{\Delta}_E^c}{|E_c|}. \quad (6)$$

When $E_c = \emptyset$ the lookahead term is zero. This is the SABRE objective of Eq. (2) with one change: SABRE weights every gate of its extended set equally, having filled that set in topological order, which fixes no notion of how far ahead a

Algorithm 1 dSABRE `route`(C, ℓ_0). An empty legal candidate set goes straight to recovery rather than waiting out the stall window, since no local SWAP can create headroom in a neighbouring core. RECOVER is the transaction of Section III-F.

$W \leftarrow$ working DAG of C; $\ell \leftarrow \ell_0$; initialise ℓ^{-1}
verify the preconditions of Theorem 1, relaying vacancies until Eq. (13) holds; initialise a checkpoint
while $W \neq \emptyset$ **do**
  Drain executable 1Q gates and adjacent intra-core front-layer 2Q gates
  if $W = \emptyset$ **then**
    break
  end if
  Form F, partition it into F_{intra} and F_{inter}, and build E if stale
  if $F_{\text{intra}} \neq \emptyset$ **then**
    for each active core c with $F_c \neq \emptyset$ **do**
      Apply the minimum-score intra-core SWAP of core c using Eq. (6)
    end for
  else
    Enumerate teleport macro-actions for F_{inter} whose landing core c' has $f_{c'} \geq \phi_1 = 2$
    Try candidates in score order; each attempt either commits atomically or restores its entry state
    if the candidate set is empty, or no candidate commits **then**
      restore the checkpoint and invoke RECOVER(W, ℓ)
    end if
  end if
  if $|W|$ decreased **then**
    replace the checkpoint and reset the stalled counter
  else if the stalled counter reaches L_{deadlock} **then**
    restore the checkpoint and invoke RECOVER(W, ℓ)
  end if
end while
return routed circuit, final layout, and metrics

gate lies, whereas $\gamma^{\text{dep}(g)}$ discounts by exactly that. Candidate evaluations are independent across active cores.

D. Inter-Core Teleportation Scoring

For an inter-core front gate $g = (q_1, q_2)$, both operands are considered as sources. For source q_1 in c_{src}, the router enumerates each adjacent core c_{next} and each link (π_s, π_d) joining them, retaining only candidates that satisfy Eq. (4). Let $p_1 = \ell(q_1)$ and let n_s be a neighbour of π_s minimising $d_{\text{intra}}(p_1, n_s)$. The macro-action clears both ports, stages q_1 to n_s, and teleports it across (π_s, π_d). Clearing a port means walking a free slot to it by intra-core SWAPs, so each core involved must hold one. The source core does, because ordinary routing leaves no core full, as Section III-F proves; Eq. (4) gives the landing core two, one of which walks to π_d to receive the arriving state while the other leaves the core non-full once it lands.

The score is

$$s = d_{\text{prep}} + c_{\text{cap}} + \Delta_F + w_e \widetilde{\Delta}_E, \tag{7}$$

where lower is better and

$$d_{\text{prep}} = d_{\text{intra}}(p_1, n_s) + d_{\text{evict}}(\pi_s) + d_{\text{evict}}(\pi_d), \tag{8}$$

$$c_{\text{cap}} = c_{\text{pen}} \max\{0, \tau - f_{c_{\text{next}}}\}, \tag{9}$$

$$\Delta_F = d^g_{\text{new}} - d^g_{\text{old}}, \tag{10}$$

$$\widetilde{\Delta}_E = \sum_{h \in E(q_1)} \gamma^{\text{dep}(h)} \left(d^h_{\text{new}} - d^h_{\text{old}}\right). \tag{11}$$

Here $d_{\text{evict}}(\pi)$ is the number of SWAPs that clear port π and is zero when π is empty, so an occupied port is priced rather than silently excluded. $E(q_1) \subseteq E$ is the set of extended-set gates having q_1 as an operand, and for a gate h with operands q_1 and q', $d^h_{\text{old}} = d_{\text{phys}}(p_1, \ell(q'))$ and $d^h_{\text{new}} = d_{\text{phys}}(\pi_d, \ell(q'))$.

Δ_F uses the same d_{phys}, and every distance term is measured on the weighted architecture graph $G_{\mathcal{A}}$—an intra-core edge costs one, an inter-core link 10—so d_{prep}, Δ_F and $\widetilde{\Delta}_E$ share one scale. Unlike Eq. (6), neither progress term is divided by a set size, because a teleport moves one operand of one gate and front-layer gates are qubit-disjoint, so exactly one front-layer term ever changes. The capacity term prices the immediate landing core, not the core holding the other operand.

The candidates generated for every gate of F_{inter} are pooled into a single ranking, so one arg min chooses which remote gate to advance as well as how to advance it. Equation (11) is an approximation adopted for efficiency in two respects: it sums only over gates involving the teleported qubit, ignoring extended-set gates the move affects through its evictions, and it scores the layout after the teleport rather than the intermediate staging and eviction layouts.

Running example: Figure 4 and Table II score the three outgoing-link candidates on the 2×3 H-grid at defaults, isolating one factor each, and show the three groups of Eq. (3) deciding between them: A wins on the cheapest staging plus the largest progress; B moves away from C_5, turning Δ_F against it; C buys A's progress but lands in a nearly-full core, where the capacity term alone reverses the decision.

E. Depth-Ordered Extended Set

This section constructs the inter-core extended set E of Eq. (11). The set is a breadth-first expansion of the remaining DAG. Gates in BFS layer k receive $\text{dep}(g) = k$; within a layer, gates sharing a qubit with a front-layer gate are emitted first. Construction stops after L gates, so every retained gate is no deeper than any omitted gate. Maintained remaining in-degrees give $O(L)$ construction time per update.

The intra-core sets E_c of Eq. (6) are breadth-first for the same reason: E_c is not built separately but selected from E, as the gates whose two operands currently reside in c, keeping the depth E has already assigned them. One construction therefore serves both scorers, and $\text{dep}(g)$ means the same thing in each. Section IV-D measures what the BFS order is worth against a topological one.

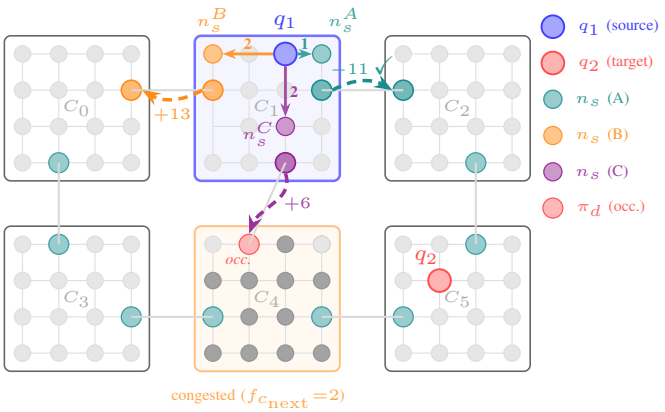


Fig. 4. How dSABRE chooses where to teleport a qubit. A single cross-core gate $g{=}(q_1, q_2)$ is pending, with q_1 in core C_1 (blue border) and its partner q_2 in C_5; the router scores the three links leaving C_1 and takes the cheapest by Eq. (7). Executing any one of them takes two steps: intra-core SWAPs first move q_1 to the staging slot n_s (count on the solid arrow), then a teleport along the link (π_s, π_d) carries it into the next core (dashed arc, annotated with the resulting score s). Candidates **A** (teal), **B** (orange) and **C** (violet) correspond row by row to Table II; C_4's orange border marks that only two of its slots are free, and the red port marked *occ.* is already occupied.

TABLE II
SCORE BREAKDOWN FOR THE THREE CANDIDATES OF THE RUNNING EXAMPLE (FIGURE 4). LOWER IS BETTER; CANDIDATE A IS SELECTED.

Candidate	next core	direction	d_{prep}	c_{cap}	Δ_F	s
A	C_2	towards C_5	1	0	-12	$\mathbf{-11}$
B	C_0	away from C_5	2	0	$+11$	13
C	C_4	towards C_5	3	15	-12	6

F. Capacity Conditions and Recovery

The ordinary score ranks candidate actions but does not ensure that a legal action always exists. TeleSABRE reports the same failure mode and leaves a SABRE-style safety valve for it to future work [10]. We therefore impose a hard capacity floor and use a recovery transaction when ordinary routing stalls. Let f_c be the number of free slots in core c, and let $F_{\text{free}} = \sum_{c=1}^{K} f_c = P - n$. Local SWAPs preserve every per-core occupancy count.

Before routing, we require

$$F_{\text{free}} \geq 2K + 1, \tag{12}$$

$$f_c \geq 2 \quad \text{for every core } c. \tag{13}$$

Throughout routing we further require

$$f_c \geq 2 \quad \text{before every teleport into } c. \tag{14}$$

Equation (14) is broader than the legality floor of Eq. (4), which governs candidate generation in ordinary routing alone: it is required of every teleport the router emits, including those inside the recovery transaction below. The two free slots form a reserve: one allows an occupied port to be evacuated, and the other allows an arriving state to be accepted. Ordinary routing need not preserve this full reserve, but it preserves at least one free slot per core; the online appendices measure some core below the reserve at 24.6% of action boundaries on the 64-qubit suite.

Algorithm 2 RECOVER(W, ℓ): the guaranteed transaction of Section III-F, which retires one blocked inter-core gate and returns with Eq. (13) restored.

1: record the entry state; on any failed precondition below, restore it and abort
2: pick a blocked inter-core front gate $g = (q_1, q_2)$, in cores a, b
3: **while** some core c_t has $f_{c_t} < 2$ **do**
4: walk a vacancy to c_t from its nearest donor // re-establish Eq. (13)
5: **end while**
6: choose $m \in \{a, b\}$ with smaller relay shortfall
7: relay vacancies until $f_m \geq 3$
8: walk the other operand to m along a shortest core path
9: SWAP within m until $\ell(q_1), \ell(q_2)$ are adjacent; execute g
10: **return** the updated W, with g retired, and ℓ

Lemma 1 (No full core)**.** *If Eqs. (13) and (14) hold, then every state reached by ordinary routing or rollback satisfies $f_c \geq 1$ for every core c.*

Proof. An intra-core SWAP does not change any f_c. A teleport increases the source-core count by one and decreases the destination-core count by one; the latter remains at least one by Eq. (14). A rollback restores a previously reached state, so the result follows by induction. □

Lemma 2 (Legal candidates commit)**.** *Suppose Eqs. (13) and (14) hold. If every intra-core coupling graph is connected, then every teleport obeying Eq. (14) can be completed atomically.*

Proof. Consider an admitted teleport from core c_s through port π_s to core c_d through port π_d. By Lemma 1, c_s contains at least one free slot. Choose any neighbour n_s of π_s. Since the coupling graph of core c_s is connected, its edge transpositions generate the full symmetric group on its vertex set. Hence intra-core SWAPs can simultaneously place the source qubit at n_s and a free slot at π_s. By Eq. (14), c_d contains at least two free slots. Connectivity similarly allows one of them to be placed at π_d, leaving at least one other free slot in c_d. The teleport consumes the free slot at π_d, so c_d remains non-full. Thus source staging, port evacuation, and the teleport are all executable, and the atomic macro-action commits. □

Recovery transaction: Recovery uses two cores, chosen for different reasons. A *donor* is any core with a free slot to spare, $f_{c_d} \geq 3$; Eq. (12) guarantees a donor exists. The *meeting core* is where the blocked gate will execute. A *relay* advances one vacancy one hop along a shortest core-graph path from the donor *nearest* the core in deficit, by teleporting a *filler*—any qubit other than the two operands of the blocked gate, which the transaction holds fixed—the opposite way.

Algorithm 2 states the transaction. For simplicity, the meeting core is selected from the two operand cores, so only one operand moves, and line 7 tops m up to three free slots so that it is back at the reserve once the arrival lands. Theorem 1 below shows that these choices always succeed; it does not claim they are cheap. The nearest donor, the operand meeting core, and the shortest-path walks are fixed to make completion provable, and a transaction spends EPR like any other action.

Theorem 1 (Termination with success)**.** *Suppose the core graph and every intra-core coupling graph are connected, every core has at least four physical sites, Eqs.* (12)–(13) *hold initially, and every teleport obeys Eq.* (14)*. Then* dSABRE *routes every gate of the input DAG and does not abort.*

Proof. F_{free} is conserved, so Eq. (12) supplies a donor in every reachable state, and by Lemma 1 no action creates a full core. Checkpoint–rollback and the invocation of recovery after L_{deadlock} stalled iterations are steps of dSABRE itself (P4 of Section III-B), not further hypotheses. It therefore suffices to show that one invocation of Algorithm 2 always retires its gate and returns in a state again satisfying Eq. (13).

Relays (line 3). Let c_t have $f_{c_t} < 2$ and take a shortest core path $c_0, \ldots, c_h = c_t$ from a donor c_0 minimising $d_{\text{core}}(c_0, c_t)$. Hop i teleports a filler from c_i to c_{i-1}, so c_{i-1} loses a slot and c_i gains one; performing the hops in the order $i = 1, \ldots, h$ keeps every core's count of free slots at or above its entry value except c_0, which ends one down but started at three. Each hop is a teleport primitive and commits by Lemma 2, its receiving core meeting Eq. (14) throughout: $f_{c_0} \geq 3$ for the donor, and for the rest $f_{c_{i-1}} \geq 2$, since Lemma 1 leaves it one free slot and the preceding hop adds another. A filler exists at every hop—the only use of the four-site hypothesis. The blocked gate is inter-core, so its operands sit in different cores and no core holds more than one of them. Were some c_i, $1 \leq i \leq h$, to hold no eligible filler, every qubit in it would be a held operand, so it would hold at most one and have $f_{c_i} \geq M - 1 \geq 3$. For an intermediate that makes it a donor with $d_{\text{core}}(c_i, c_t) < d_{\text{core}}(c_0, c_t)$, contradicting the choice of c_0; for c_t it contradicts $f_{c_t} < 2$. Each repetition reduces the total deficit $\delta = \sum_c \max\{0, 2 - f_c\}$ by one and creates no new deficit, and Lemma 1 makes every summand 0 or 1, so $\delta \leq K$ and at most K vacancy walks, of at most D_K relays each, restore Eq. (13).

Rendezvous (lines 6–7). Take $m = a$. Every core now holds two free slots and $F_{\text{free}} \geq 2K + 1$, so some core holds three; one vacancy walk from it, by the argument above, leaves $f_a \geq 3$ without pushing any core below two. Walk the operand in b to a along a shortest core path. Every core on it is at the reserve before the operand arrives, and a is at three, so each hop meets Eq. (14). An intermediate core dips to one free slot while it holds the operand—enough to clear its outgoing port—and returns to two when the operand leaves; b ends one up and a one down, at $f_a \geq 2$. So Eq. (13) holds when the walk ends: this is why line 7 tops a up first rather than repairing afterwards. Intra-core connectivity then makes the operands adjacent by SWAPs, which change no count, and the gate executes.

So the transaction strictly decreases $|W|$ and preserves Eq. (13). Between two decreases of $|W|$, the ordinary loop performs at most L_{deadlock} stalled iterations before recovery is invoked. Strong induction on $|W|$ proves that the router reaches $|W| = 0$ in finitely many steps and never aborts. □

Satisfying the hypotheses: Every device and workload evaluated here satisfies the hypotheses with substantial margin. Connectivity is needed to route at all: without it a core cannot bring its own qubits together, and the circuit splits into independent components. The size condition asks a core to hold four physical sites, the smallest used here holding 16. Equation (13) need not be arranged by the user at all: Section III-G shows how to build a layout that reserves at least two sites in every core, so the condition holds by construction. Equation (14) is enforced rather than assumed—by the legality floor on ordinary teleports, and by the reserve on the ones recovery emits. That leaves Eq. (12) as the only condition on the workload, and it is loose: the device may be filled to $1-(2K+1)/P$, which is 86% for the 16-site cores used here and 91–92% for the 25-site ones. The tightest instance in this paper, the 64-qubit suite on the H-grid, has 32 free slots where 13 suffice. A circuit that violates Eq. (12) is rejected before routing rather than part-way through.

G. Initial Layout

Theorem 1 starts from a layout that already holds the per-core reserve of Eq. (13); this section constructs one. The default layout bootstraps from Qiskit's `SabreLayout` on the full architecture graph, but reserves slots in every core. It withholds the k minimum-degree sites that are farthest, by total shortest-path distance, from the rest of their core—corners on grids and leaves on heavy-hex cores. The value $k \geq 2$ is chosen so that the resulting layout satisfies Eq. (13); the reservation is per core rather than a whole-device quota. Forward, backward, and final-forward passes are run, retaining the better forward route by EPR count. Three `SabreLayout` seeds are used, so one compilation performs nine routes in all, of which the reported one is the best forward pass.

H. Complexity

Let N be the number of gates, and let L, the intra-core and core-graph degrees, and the number of links joining one adjacent pair of cores be constants. Distance and shortest-path tables are precomputed once per architecture, in $O(KM^2D_M + K^2(D_K + \log K))$ time and space, and are not charged below; d_{phys} is composed on demand by Eq. (1) rather than materialised, which avoids the $\Theta(P^2)$ table a whole-chip all-pairs search would build.

Three costs then compose. Maintaining the extended set is $O(L)$ per retired gate, hence $O(N)$ over a route. One routing iteration is dominated by teleport selection: front-layer gates are qubit-disjoint and the core degree and link multiplicity are bounded, so a candidate set has $O(n)$ members, each of which may cost the current implementation an $O(M)$ scan of its landing core, giving $O(M(K+n))$ with the free-slot census. Recovery follows after at most $L_{\text{deadlock}} = 4D_K(D_M + 1)$ stalled iterations and retires a gate, so ordinary iterations number $O(ND_K(D_M+1))$ and transactions at most N, each costing $O(KM(K+D_K))$ to restore the reserve and move an operand. Summing the three and simplifying with $K = O(n)$ and $D_K \leq K - 1$,

$$T_{\text{total}} = O\big(NM\left(D_K D_M n + K^2\right)\big). \tag{15}$$

For grid-of-grids architectures $D_K = O(\sqrt{K})$ and $D_M = O(\sqrt{M})$, so $D_K D_M = O(\sqrt{P})$ with $P = KM$, giving $T_{\text{total}} = O(NM(\sqrt{P}n + K^2))$. This is a deliberately loose

common bound in two respects: the K^2 term is the recovery contribution, charged once per gate even though Section IV-D measures recovery firing on a small fraction of them, and the $O(M)$ scans are an artefact of the implementation that maintained free-slot and donor structures would remove. The online appendices derive each term in full and record the structural properties the derivation uses and the measurements that check it against the implementation.

IV. Experimental Evaluation

A. Setup

Goal. We evaluate whether dSABRE reduces EPR consumption while retaining practical compile time and completing instances on finite-capacity multi-core devices. Our primary comparison is with TeleSABRE [10], because it uses the same physical architectures, counts local SWAPs, and supports one-EPR teledata moves. It is also the strongest baseline of its family: it reports a 28% geometric-mean reduction in inter-core state transfers over Hungarian Qubit Assignment [4], [16], the partitioning-based multi-core mapper it measures itself against. The comparison with `pytket-dqc` is reported separately: its EPR unit is the same, but an EPR pair there represents different work and assumes a different machine model.

Benchmarks and architectures. We use 21 MQT Bench circuits [17], transpiled to the IBM native basis with Qiskit optimisation level 3; measurements and barriers are removed. The 25-qubit suite contains AE, QFT, QNN, Random, GHZ, and Graphstate. The 36-qubit suite contains BV, DJ, QAOA, QPEexact, VQE-SU2, and W-state. The 64-qubit suite contains AE, QFT, QNN, Random, GHZ, Graphstate, QPEexact, QAOA, and a 16-bit Multiplier. The scalability study uses two families, QFT and QPEexact, at 100, 200, and 360 logical qubits.

The 25- and 36-qubit suites run on a 2×2 B-grid with four 4×4 cores (64 physical qubits and four inter-core links). The 64-qubit suite runs on a 2×3 H-grid with six 4×4 cores (96 physical qubits and seven links). These are the TeleSABRE benchmark topologies. We additionally use four 27-qubit heavy-hex cores connected as a ring or star, and 5×5 grid cores for the scalability experiments. Every core graph and every intra-core coupling graph is connected, as Section II-C assumes. Counting a circuit once per architecture on which it is routed—the 21 suite circuits, the six scalability instances, and the nine 64-qubit circuits on each of the two heavy-hex networks—the evaluation covers 45 instances.

Protocol. Unless stated otherwise, dSABRE uses the fixed parameters in Table I and its default capacity-reserving initial layout. dSABRE and TeleSABRE each report the best result from three seeds, following their standard use; `pytket-dqc` reports the best of five. We report EPR pairs and local SWAPs. For TeleSABRE, EPR is the sum of teledata and telegate operations. A tool is reported as non-convergent on an instance when none of its seeds returns a valid routing. For TeleSABRE this is its own verdict rather than a limit we impose: every seed exhausts its ten internal attempts at the tool's own deadlock criterion, well inside the wall-clock budget of 300 s per seed (900 s on the scalability series). A failed run is reported explicitly and is excluded from a baseline-only geometric mean; the corresponding dSABRE mean over all completed circuits is also shown. Per-seed results and commands are provided in the supplementary material. All measurements ran single-threaded on one machine (Apple M2, 16 GB, macOS 26.5). dSABRE is pure Python (3.13, Qiskit 2.3) and TeleSABRE a compiled C++ binary, so the times of Section IV-C weigh an interpreted implementation against a native one; `pytket-dqc` 0.0.1 (pytket 2.16, KaHyPar 1.3.5) is given 900 s per distributor per circuit.

B. Main Comparison with TeleSABRE

Table III gives the main results. dSABRE lowers geometric-mean EPR consumption by 50.8%, 62.0%, and 49.1% on the 25-, 36-, and 64-qubit suites, respectively, on the circuits completed by both routers. It also uses fewer geometric-mean local SWAPs in every suite. At 64 qubits, TeleSABRE fails on Random in all three attempts, whereas dSABRE completes it using 732 EPR pairs. The result is therefore not only an average EPR reduction: dSABRE also completes one additional circuit under the same architecture and run budget. Over the whole evaluation, dSABRE completes 45 of the 45 circuit–architecture instances and TeleSABRE 40 of 45, under the run budget of Section IV-A. These figures are best-of-three for both routers; taking the median seed instead leaves the reductions at 50.3%, 59.9%, and 42.9%, so the comparison does not rest on the best-of-N convention.

C. Robustness, Scale, and Runtime

Topology. The core here is a 27-qubit IBM heavy-hex tile, which shares none of the grid's regularity—degrees over $\{1, 2, 3\}$, six degree-1 leaves, and diameter 12 against the 4×4 grid's 6—and four such tiles are wired as a ring (4 links) and as a star (3 links, with a hub every inter-core route must cross). Nothing is specialised for either: the corner reservation of Section III-G reads vertex degree and remoteness, so it selects leaf qubits where it selected grid corners. On the ring both routers complete all nine 64-qubit circuits and dSABRE reduces EPRs by 52.0% against TeleSABRE; on the star the reduction is 41.5%, with TeleSABRE again failing on Random where dSABRE uses 475 EPR pairs. The improvement is therefore not specific to regular grid cores or a mesh core graph.

Scale. We scale two circuit families, QFT and QPEexact, while holding core occupancy approximately constant: 100, 200, and 360 logical qubits use 6, 12, and 20 5×5 cores, respectively. On QFT, dSABRE uses 201 EPR pairs at 100 qubits against TeleSABRE's 248, and 766 against 1232 at 200 qubits; at 360 qubits TeleSABRE does not converge in any of three runs, while dSABRE completes at 1890. On QPEexact the margin is larger and the baseline gives out earlier: 246 EPR pairs against 586 at 100 qubits, and no convergence at either 200 or 360 qubits, where dSABRE completes at 896 and 1938. Three of the five non-convergent instances in this paper are therefore these large routes; the other two are Random at 64 qubits, on the H-grid and on the heavy-hex star. These cases show that dSABRE's capacity-aware routing

TABLE III
MAIN COMPARISON WITH TELESABRE. EPR IS TELEDATA PLUS TELEGATE FOR TELESABRE AND TELEDATA FOR DSABRE. NEGATIVE Δ FAVOURS DSABRE. *TELESABRE FAILS TO CONVERGE; EVERY GEOMETRIC MEAN AND ΔEPR USES ONLY THE CIRCUITS BOTH ROUTERS COMPLETE, THE DSABRE-ONLY MEAN BEING A ROW OF ITS OWN.

		TeleSABRE		dSABRE		
Circuit	CX	EPR	SWAP	EPR	SWAP	ΔEPR (%)
25 qubits, 2×2 B-grid						
AE	558	26	292	23	288	-11.5
GHZ	24	4	36	1	4	-75.0
Graphstate	25	13	47	2	15	-84.6
QFT	580	46	405	33	332	-28.3
QNN	1223	54	659	47	661	-13.0
Random	1124	271	1248	181	1208	-33.2
Gmean (6)		31.1	234	15.3	129	**−50.8**
36 qubits, 2×2 B-grid						
BV	17	6	39	1	13	-83.3
DJ	35	4	63	3	23	-25.0
QAOA	1200	268	1165	134	1017	-50.0
QPEexact	1019	190	883	62	612	-67.4
VQE-SU2	105	28	134	9	45	-67.9
W-state	70	13	79	6	40	-53.8
Gmean (6)		27.6	173	10.5	83	**−62.0**
64 qubits, 2×3 H-grid						
AE	1962	353	1494	165	1325	-53.3
GHZ	63	16	132	6	39	-62.5
Graphstate	64	62	281	15	88	-75.8
QFT	1966	312	1711	180	1480	-42.3
QNN	8126	889	5471	539	5260	-39.4
Random*	1627	—	—	732	3603	—
QPEexact	2139	289	1706	187	1576	-35.3
QAOA	3920	956	4189	555	3734	-41.9
Multiplier	13040	1601	9654	1285	8969	-19.7
Gmean (8 common)		284.5	1564	144.7	1081	**−49.1**
Gmean (9 dSABRE)		—	—	173.3	1236	—
Scalability, 5×5 cores throughout, core count $6 \to 12 \to 20$						
QFT 100q	3420	248	2414	201	2452	−19.0
QFT 200q	7220	1232	7089	766	6482	−37.8
QFT 360q*	13300	—	—	1890	15606	—
QPEexact 100q	3579	586	3485	246	2836	−58.0
QPEexact 200q*	7579	—	—	896	6834	—
QPEexact 360q*	13979	—	—	1938	16090	—

TABLE IV
ABLATION ON THE 64-QUBIT SUITE. ALL SETTINGS OTHER THAN THE NAMED COMPONENT ARE UNCHANGED. EVERY Δ COMPARES AGAINST FULL DSABRE ON THE SAME CIRCUITS: THE LOWER BLOCK IS EVALUATED ON A SUBSET, SO ITS *ref.* COLUMN REPEATS FULL DSABRE'S MEAN OVER THAT SUBSET AND ITS VALUES MUST NOT BE READ AGAINST THE NINE-CIRCUIT BLOCK ABOVE.

Configuration	circ.	gmEPR	ref.	Δ (%)	aborts
All nine circuits					
Full dSABRE	9	173.3	—	—	0
Topological set, not BFS	9	194.8	173.3	+12.4	0
No capacity score (c_{pen}=0)	9	189.5	173.3	+9.4	0
No legality floor (Eq. (4))	9	173.6	173.3	+0.2	0
Strict floor (ϕ_1=3)	9	201.5	173.3	+16.3	0
Common-subset controls					
Neither floor nor score	**7**	**207.9**	110.7	**+87.8**	**2**
TeleSABRE layout, not ours	8	189.4	144.7	+30.9	0

The "neither" row is compared on the seven circuits it completes; the TeleSABRE-layout row on the eight for which a TeleSABRE layout exists.

and recovery framework remains operational as the core graph grows. They are not evidence about *why* the baseline stops: non-convergence there could equally reflect search behaviour, candidate cycling, or implementation limits, and we do not attribute it to capacity handling.

Runtime. On the eight 64-qubit circuits completed by TeleSABRE, the geometric-mean compilation time is 1.9 s for dSABRE, 4.7 s for TeleSABRE, and 107 s for `pytket-dqc`; on the largest circuit, the 13 040-CX Multiplier, the three take 23.7, 136.3, and 1393 s. The gap is not uniform—on QNN at 8126 CX the two routers are within 0.2 s—and the tools use different search procedures, so these are end-to-end practical costs rather than an operation-level comparison. Timing includes the complete routing or distribution attempt; for dSABRE that is layout generation and all nine routes. Within one architecture, compile time tracks gate count: a log–log fit of per-seed time against CX count gives an exponent of 0.97 (R^2=0.95) over the nine 64-qubit circuits, and 0.92 and 1.21 over the 36- and 25-qubit suites. Each fit holds the device fixed, so it measures growth in circuit size alone; the scalability series, where the machine grows with the circuit, is fitted separately in the online appendices.

D. Ablation and Completion

Table IV isolates the main design choices on the complete 64-qubit suite. Replacing the BFS depth-ordered extended set with a topological-order set costs 12.4% in geometric-mean EPR, and removing the capacity score 9.4%. Removing both the score and the legality floor of Eq. (4) aborts QNN and Multiplier and costs 87.8% over the seven circuits every configuration completes, whereas removing the floor alone costs 0.2% and no aborts: the penalised band $f_{c_{\text{next}}} < \tau$=3 contains the region $f_{c_{\text{next}}} < \phi_1$=2 that the floor forbids, so while the score is present the router seldom proposes an illegal landing. That is a fact about these instances, not a licence to drop the floor—Theorem 1 needs Eq. (14) of *every* teleport, and a penalty, however heavy, can be outvoted where an admission test cannot. Tightening the floor to ϕ_1=3 costs 16.3% instead; the online appendices record what the rows without the floor also switch off.

Table V counts how often the guarantee is invoked. A *transaction* is one invocation of Algorithm 2 and retires exactly one gate; a *relay* is one filler teleport that walks a vacancy one hop, issued inside the transaction (lines 3 and 7 of Algorithm 2) and by the start-up repair of line 2 of Algorithm 1. The layout of Section III-G satisfies Eq. (13) by construction, so that repair never fires on a forward pass; it fires when the backward and final-forward passes inherit the preceding pass's final layout, which ordinary routing may leave below the reserve. That is the whole of the relay count in the first two rows. The guarantee is rarely used on the reported small and medium instances: no selected route among the 21 circuits at 25–64 qubits invokes recovery. It becomes relevant at scale, where recovery retires 6, 10, and 66 gates over the reported 100-, 200-, and 360-qubit routes. Across all 1074 transactions no asserted recovery precondition failed, and none of the 1.08 million candidates scored was rejected at execution time, as

TABLE V
GATES RETIRED BY THE GUARANTEED TRANSACTION OF SECTION III-F, AND RELAY TELEPORTS, OVER EVERY SUITE OF TABLE III. THE TWO TRANSACTION COLUMNS ARE NESTED—*search* COVERS ALL NINE ROUTES PER CIRCUIT, OF WHICH THE *reported* ONE IS A MEMBER—WHILE RELAYS ARE TELEPORTS, COUNTED OVER THOSE SAME NINE ROUTES BUT NOT TIED TO A TRANSACTION.

			Transactions		Relays
Suite	Cores	Circuits	Reported	Search	(search)
25q	4	6	0	0	4
36q	4	6	0	0	5
64q	6	9	0	42	145
100q	6	2	6	73	250
200q	12	2	10	214	756
360q	20	2	66	745	3822
Total		**27**	**82**	**1074**	**4982**

TABLE VI
`PYTKET-DQC` AGAINST DSABRE, BY SUITE. `PYTKET-DQC` RUNS ITS DEFAULT COMPUTATION ON A NETWORK MORE PERMISSIVE THAN THE DEVICE (SECTION IV-E). SUITE FIGURES ARE GEOMETRIC MEANS OVER THE CIRCUITS OF TABLE III. *Unbuildable* COUNTS THE DISTRIBUTIONS THAT PROVABLY EXCEED THE DEVICE'S COMMUNICATION CAPACITY: EXACT AT 25 AND 36 QUBITS, A LOWER BOUND AT 64, WHERE MULTIPLIER'S CAPACITY SEARCH HAD NOT TERMINATED WHEN ITS BUDGET EXPIRED AND IS LEFT UNDETERMINED RATHER THAN COUNTED.

Suite	`pytket-dqc` e-bits	dSABRE EPR	Δ (%)	unbuildable
25 logical qubits	34.9	15.3	−56.1	4/6
36 logical qubits	12.2	10.5	−13.8	2/6
64 logical qubits	150.2	173.3	+ 15.3	6/9

Lemma 2 guarantees. Hence the guarantee is a fallback rather than the source of the main-suite EPR gains.

A shared-layout control further separates layout from routing. Starting dSABRE from TeleSABRE's initial layout increases its 64-qubit EPR geometric mean by 30.9%, but dSABRE still improves on TeleSABRE by 33.4%.

The defaults of Table I are not tuned per instance. In the one-at-a-time sweeps of the supplementary material, every default is the best value swept on the 64-qubit suite and within 2.5% of the best at 25 qubits, and the advantage persists as the teleport-to-SWAP cost ratio varies from 10 to 100.

E. Comparison with `pytket-dqc`

This comparison must be interpreted as a cost-model study, not a like-for-like router comparison. `pytket-dqc` [6], [12] uses hypergraph partitioning and gate teleportation, allowing one EPR pair to serve several gates when it remains live. We run COVEREMBEDDINGSTEINERDETACHED, its strongest embedding distributor, falling back to PARTITIONINGHETEROGENEOUS where that returns no distribution in budget (at 64 qubits: QNN, Multiplier, Random), and report the best of five seeds. Its input network is more permissive than the evaluated device: all qubits are available for data and the number of live link qubits per core is unbounded.

Under those assumptions (Table VI), dSABRE uses 56.1% and 13.8% fewer EPR pairs on the 25- and 36-qubit suites; at 64 qubits dSABRE uses 15.3% more, 173.3 EPR pairs against `pytket-dqc`'s 150.2 e-bits. However, 4/6, 2/6, and at least 6/9 of the respective distributions provably cannot be materialised within the evaluated communication-port capacity—25-qubit AE requires 20 simultaneous link qubits at a core where the device provides two.

Entanglement lifetime is a separate assumption. Re-scoring the same `pytket-dqc` distributions with a maximum of three gates per EPR pair changes the 64-qubit result to a 59.6% dSABRE advantage. The reversal shows where gate teleportation is valuable—circuits with long reusable interaction groups—but also that its reported EPR count depends on maintaining entanglement across more than ten gates per pair. dSABRE's teledata count is independent of that lifetime because each pair is consumed immediately.

Takeaway: The controlled TeleSABRE comparison establishes the main result: capacity-aware teledata routing reduces EPR use and improves completion on the tested finite-capacity architectures. The `pytket-dqc` study identifies a complementary limitation—gate-teleportation amortisation can be preferable when long-lived entanglement and sufficient communication memory are available—and should not be read as a direct win or loss without those qualifications.

V. RELATED WORK

Distributed compilation combines qubit allocation, communication selection, routing, and scheduling. This section positions dSABRE at the routing stage; Section I gives the broader pipeline.

SABRE-style distributed routing. TeleSABRE [10] is the closest prior router and the main experimental baseline. Both methods extend a SABRE-style, distance-and-lookahead search to multi-core hardware. TeleSABRE selects from local SWAP and inter-core candidates in one pooled search and uses shortest paths on a contracted communication graph. dSABRE differs in three respects. It prices *availability*: a teledata move is scored as a macro-action including port staging and any required eviction, rather than as a bare hop an occupied port would delete. It *selects* by a priority rule: no teleport is scored while any same-core front-layer gate remains, including those whose operands still need local SWAPs—under TeleSABRE's pooled arg min, 59.5% of its 64-qubit communication operations fire while a same-core front gate is still waiting. And it charges *capacity* at a different point in the search: TeleSABRE folds it into the shortest path it computes between the two operands, as a penalty on the weights of a core near its limit, so capacity steers which route is taken, whereas dSABRE scores it on each candidate's own landing core and, separately from the score, admits no candidate that breaks Eq. (4). TeleSABRE additionally considers telegate operations, so the two action sets differ as well.

DMapS [11] is a concurrent end-to-end approach that combines KaHyPar-based partitioning with a SABRE-derived router. Its cross-core primitive is a two-EPR remote SWAP, whereas dSABRE teleports one logical state into a layout-reserved free slot, and that choice propagates into a different device model. A remote SWAP exchanges two states across

the link and a one-EPR remote CNOT moves nothing, so both leave each chip's occupancy unchanged. DMapS therefore never needs a free slot: a chip may stay full for the whole route, and no capacity condition is maintained during routing. Its coupling graphs omit communication qubits accordingly, so every physical qubit carries data, an inter-chip link counts one hop in its qubit distance matrix exactly as an intra-chip edge does, and no remote operation can be blocked by an occupied port. These are modelling choices, not errors, but they leave the two EPR counts comparable only with the assumptions stated. Over the nine-circuit 64-qubit suite dSABRE uses $1.43\times$ fewer EPR pairs in geometric mean, DMapS winning only on GHZ and Graphstate, which KaHyPar partitions almost perfectly; the per-circuit head-to-head is in the online appendices.

Device model. The three comparisons differ in what they assume the hardware provides, and dSABRE's assumptions are the most conservative. Following TeleSABRE, it represents communication ports explicitly, treats a port as an ordinary slot that must be evacuated before it can carry a state, gives an inter-core link a cost of its own, bounds every core's capacity, and consumes each EPR pair at once. The other two relax exactly these points: `pytket-dqc` allows unboundedly many live link qubits per core and entanglement that survives a whole group of gates, and DMapS omits the ports and with them the capacity question. Modelling more of the device is not demanding more of it: dSABRE asks only for a connected coupling graph, the one to four ports per core the evaluated devices already have, two spare slots per core and one more on the device (Eqs. (12)–(13)), and no entanglement storage at all, whereas the alternative models assume communication memory or occupancy-preserving remote operations the evaluated architecture does not represent. Extra ports, heterogeneous link costs, or a non-uniform core graph enter through the same distance tables and the same d_{prep} term, leaving the score and the completion argument unchanged.

Allocation, communication, and scheduling. Hypergraph-based methods assign qubits to cores by minimising cut communication [5], [6]. They are complementary to dSABRE: their output can supply an initial allocation, while dSABRE resolves the local movement and capacity constraints that arise during execution. Finally, time-aware DQC compilers schedule entanglement generation and remote operations subject to link and buffer constraints [18], [19].

VI. Future Work

Telegate and communication bursts. Scoring a telegate is not the hard part. A telegate executes a remote gate without relocating either operand, so it consumes no destination capacity and enters Eq. (7) with $c_{\text{cap}} = 0$, keeping only the port staging d_{prep} its two link qubits need; its progress term is the front-layer gate it retires outright rather than a distance reduction. Implemented that way and allowed to compete with teledata, it costs 93.5% more EPR on the 64-qubit suite, and the best of seventeen scoring variants we swept is still 1.0% worse than teledata alone. Amortisation is the reason, and it is measurable. One pair buys the moved qubit a residence in its new core, over which it serves 15.75 of that qubit's two-qubit gates on average across this suite, with a median of 8, and 83.4% of moves serve two or more; an ungrouped telegate serves exactly one, so it can only compete on the remaining sixth. A telegate therefore pays only in grouped form, for which the router must expose commutation-derived bursts, choose the beneficiary core, and hold entanglement for the group's duration. AutoComm [20] and QuComm [21] give pre-processing directions, and lifetime-bounded grouping in [22] shows that a burst is worth taking only when its entanglement survives the hardware limit. The measurement is in the online appendices.

Scheduling, fidelity, and interconnect co-design. This paper minimises EPR consumption rather than makespan or output error, and the trace it emits is unscheduled. A scheduling pass should batch teleports whose links, ports, and staging operations do not conflict, and should account for entanglement-generation rate, storage lifetime, and hardware error rates; existing time-aware compilers [18], [19] provide starting points. The objective should stay explicitly multi-objective, since EPR count, circuit depth, and fidelity trade off in hardware-dependent ways. The interconnect is the other free variable. The evaluated devices give each pair of adjacent cores a single link and each core one to four ports, which is what the baseline architectures provide rather than a limit of the method: Section III-D already enumerates every link joining two cores as its own candidate, though on the evaluated devices that loop never has more than one link to weigh. Widening it should relieve the availability term d_{prep} first, since a second link turns an occupied port into an alternative rather than an obstacle, and should interact with the capacity mechanism, because a core with more ports has more ways to spend and to recover a free slot. Finding where the return on added links falls off against its hardware cost is a useful input to interconnect co-design [21], [23], [24].

VII. Conclusion

This paper presented dSABRE, a SABRE-style teledata router for multi-core distributed quantum computers. Its design principle is to treat an inter-core move as a complete macro-action rather than a local SWAP with a larger edge cost, scoring the routing progress it buys—front-layer distance reduction and DAG-depth-discounted lookahead—against the two costs specific to a directional teleport, port staging and destination capacity. Capacity enters twice, and the separation is the point: as a graded score term that ranks candidates, and as a hard legality floor that, with a bounded recovery transaction, makes dSABRE provably complete—every gate routed, no abort at saturation—under checkable pre-routing conditions.

On 21 MQT Bench circuits at 25–64 logical qubits dSABRE reduces geometric-mean EPR consumption by 49–62% against TeleSABRE under the shared teledata cost model, completes all 45 evaluated circuit–architecture instances against the baseline's 40, and compiles a 360-qubit QFT on twenty cores in 48 s. Removing both capacity mechanisms costs 87.8% and leaves two instances unrouted: the principal advance is not the number of heuristic terms but the separation of resource-aware ranking from an explicit legality condition and a completion-preserving recovery. Recovery itself is a safeguard, not an

optimisation—no reported route at 25–64 qubits invokes it, and from 100 to 360 qubits it retires 6 to 66 gates.

These reductions are in one resource, under one communication model. The direct comparison is with TeleSABRE, which shares teledata and local-SWAP accounting; remote-SWAP and telegate-based tools need their own capacity and entanglement-lifetime assumptions stated before their counts can be read against ours. The emitted traces are also unscheduled, so a lower EPR count does not by itself imply a shorter makespan or a higher-fidelity result. Joint selection between teledata and grouped telegate operations, followed by time- and fidelity-aware scheduling of the resulting trace, is the natural next step.

Artefact availability: The dSABRE implementation, the routed-circuit benchmark suites, per-circuit JSON results, and the supplementary material are available at https://github.com/ebony72/dsabre.

Acknowledgments

This work was partially supported by the Australian Research Council (Grant No. DP250102952 and DP220102059). The author used Anthropic's Claude (via Claude Code) to assist with refactoring parts of the implementation, authoring benchmark scripts, drafting figure sources, and copy-editing. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[3] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[4] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[5] P. Andres-Martinez and C. Heunen, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.

[6] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[7] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[8] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2024.

[9] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] T. Luo, Y. Zheng, Y. Deng, and X. Fu, "DMapS: End-to-end qubit mapping and routing for distributed quantum computing architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025, code: https://github.com/RoccoLoter/DMapS.

[12] P. Andres-Martinez, T. Forrer, D. Mills, J.-Y. Wu, L. Henaut, K. Yamamoto, M. Murao, and R. Duncan, "Distributing circuits over heterogeneous, modular quantum computing network architectures," *Quantum Science and Technology*, vol. 9, 2024, arXiv:2305.14148.

[13] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Physical Review A*, vol. 52, no. 5, pp. 3457–3467, 1995.

[14] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[15] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "LightSABRE: A lightweight and enhanced SABRE algorithm," 2024. [Online]. Available: https://arxiv.org/abs/2409.08368

[16] P. Escofet, A. Ovide, C. G. Almudever, E. Alarcón, and S. Abadal, "Hungarian qubit assignment for optimized mapping of quantum circuits on multi-core architectures," *IEEE Computer Architecture Letters*, vol. 22, no. 2, pp. 161–164, 2023.

[17] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[18] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.

[19] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.

[20] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[21] A. Wu, Y. Ding, and A. Li, "QuComm: Optimizing collective communication for distributed quantum computing," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023.

[22] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, "Optimized compilation for distributed quantum computing," 2026, arXiv:2602.24062.

[23] P. Escofet, S. B. Rached, S. Rodrigo, C. G. Almudever, E. Alarcón, and S. Abadal, "Interconnect fabrics for multi-core quantum processors: A context analysis," in *Proceedings of the 16th International Workshop on Network on Chip Architectures (NoCArc)*, 2023, arXiv:2309.07313.

[24] J. Liu, A. Zang, M. Suchara, T. Zhong, and P. D. Hovland, "Hardware-software co-design for distributed quantum computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–6.

Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.